Quack Wiki Flask Application - Code Documentation

Quack Wiki Flask Application - Detailed Code Documentation

Generated from the current main.py source file.

Document purpose

This document explains the main Flask application and gives detailed commentary for the larger systems: startup/bootstrap, authentication and authorization, safe redirects, markdown rendering, article visibility, article and revision workflows, dashboard filtering, role management, archiving, uploads, and public article rendering.

Important note: line numbers refer to the current repository file main.py. The source code currently stores SECRET_KEY as a literal string on line 33; in a real deployment, that should be moved to an environment variable.

1. High-level architecture

main.py is the central controller for Quack Wiki. It configures Flask and SQLAlchemy, initializes Flask-Login, creates required folders and database rows at startup, defines helper functions, and declares the public, authenticated-user, writer, and admin routes.

- Models used: User, Article, ArticleRevision, SiteSettings, and db from models.py.
- Forms used: RegisterForm, LoginForm, ProfileForm, PasswordChangeForm, ArticleForm, RoleForm, and SiteSettingsForm from forms.py.
- Main workflow: users create articles; articles start pending; normal-user edits become pending revisions; trusted editors can update live articles; admins and writers can approve/decline content.
- Public visibility rule: an article must be approved, not archived, and attached to an active user.

```
Layer | Main responsibility | Important functions / routes
Application setup | Configure Flask, database, uploads, folders, startup cleanup | Lines 1-121, cleanupunusedimages(), ensureuserprofilecolumns(), getsite_settings()
Authentication | Register, login, logout, session loading, block archived users | register(), login(), logout(), load_user()
Authorization | Protect routes by role and ownership | rolerequired(), canedit_article(), route decorators
Content rendering | Convert safe markdown and infobox text into template data | inlinemarkdown(), rendersimplemarkdown(), parseinfoboxdata(), renderarticlepage()
Article lifecycle | Create, edit, approve, decline, archive, restore | createarticle(), editarticle(), approve(), decline(), archiveownarticle()
Revision lifecycle | Store pending edits and apply them after review | saveorreplacependingrevision(), applyrevision(), approverevision(), decline_revision()
Dashboards | Search, filter, sort, approve, archive, manage users and settings | dashboardusers(), dashboardarticles(), dashboardupdates(), dashboardsettings()
Public browsing | Home, search, article lists, tag-backed pages, author profiles | index(), articles(), article(), about(), tagged_article(), authors()
```

2. Imports, constants, and app configuration

Lines | What those lines do / why they matter

1-5 | Module docstring explains that main.py owns app setup, routes, rendering, uploads, and SQLite compatibility work.

7-16 | Standard-library and SQLAlchemy imports. These support timestamps, HTML escaping, filesystem work, regex parsing, identity normalization, redirect parsing, query helpers, exception handling, decorators, and UUID file names.

18-27 | Flask, Flask-Login, SQLAlchemy text SQL, Werkzeug upload/security helpers, project forms, models, and seed data are imported.

29-30 | Defines a 50 MB global request upload limit and converts it to bytes for Flask.

32-39 | Creates the Flask app and configures the secret key, SQLite database, SQLAlchemy behavior, maximum content length, and upload folders.

41 | Binds the shared SQLAlchemy db object to this Flask app.

3. Startup and maintenance helpers

Lines | What those lines do / why they matter

44-81 | cleanupunusedimages() collects all image paths still referenced by articles, revisions, profiles, and site settings, then deletes unreferenced files from managed upload folders. This prevents abandoned uploads from accumulating.

84-95 | ensureuserprofilecolumns() inspects the SQLite user table with PRAGMA tableinfo(user) and adds profile columns when an older database is missing them. This is a lightweight compatibility migration.

98-105 | getsitesettings() returns the singleton SiteSettings row with id=1, creating it on first run so templates and settings routes always have a row to read.

108-117 | Startup app context creates tables, creates upload folders, runs the SQLite compatibility migration, ensures site settings exist, seeds initial users, and cleans unused images.

119-121 | Initializes Flask-Login and sets the login route used when protected pages require authentication.

4. Request handlers, login sessions, and role-based authorization

Lines | What those lines do / why they matter

126-130 | handleuploadtoo_large() catches oversized request uploads and redirects with a friendly flash message instead of showing a raw Flask error.

133-146 | injectsitesettings() injects site_settings and dashboard pending counters into every template. Counters are only calculated for admins and writers.

149-155 | loaduser() restores currentuser from the Flask-Login session. Archived users return None, blocking their sessions.

158-171 | rolerequired() is a decorator factory. It wraps a route with loginrequired, then checks that current_user.role is in the allowed role list. Failed checks abort with 403.

Why this group is complex: rolerequired() returns a decorator, and that decorator returns the actual route wrapper. This lets routes declare role access cleanly with @rolerequired(['admin']).

5. Safe redirect protection

Lines | What those lines do / why they matter

174-180 | issaferedirect_url() rejects empty targets, resolves relative URLs against the current host, and only allows http/https redirects that stay on the same host. This blocks open redirect bugs.

183-188 | safenexturl() reads next from query/form data and returns it only when safe; otherwise it falls back to a named endpoint.

191-196 | dashboardreturnurl() keeps dashboard filters after POST actions by checking hidden form next first, then request.referrer, while still using the safe redirect guard.

6. Safe limited markdown renderer

Lines | What those lines do / why they matter

201-222 | inlinemarkdown() handles inline code, links, bold, and italic. It stashes escaped code spans, escapes the rest of the text, converts only known markdown patterns, then restores the safe code HTML.

225-329 | rendersimplemarkdown() renders a safe block-level markdown subset: headings, paragraphs, blockquotes, ordered lists, unordered lists, fenced code blocks, and inline formatting. It does not allow raw HTML.

239-260 | Nested helpers close lists, flush paragraphs, and flush code blocks. Keeping them nested makes the state local to this renderer.

262-329 | The main parsing loop processes each line, changes state for code/list blocks, creates escaped HTML blocks, flushes unfinished state at the end, and joins the blocks into one HTML string.

Key security idea: user article content is treated as text. The renderer escapes raw content and creates only a small known set of HTML tags.

7. Article identity, visibility, search, and reserved pages

Lines | What those lines do / why they matter

334-338 | articlepublicurl() sends articles tagged page:about-us to /about; all other articles use /article/<title>.

341-344 | normalize_identity() removes accents, lowercases text, and strips non-alphanumeric characters so legacy author names can match current users.

347-371 | finduserforarticleauthor() resolves an article author string to an active user by exact username first, then normalized username/display-name matching.

374-383 | articleispublic() and canopenarticle() require approved status, non-archived article state, and an active resolved author.

386-394 | staticpagetag() and articlehastag() normalize reserved page tags and check tag lists case-insensitively.

397-422 | tagsearchneedle() and articlematchessearch() support normal text search and #tag search. #tag searches intentionally look at tags only.

425-437 | usermatchespublic_search() searches public user fields only and does not search email addresses.

440-474 | activeusersbyusername(), publicarticlesforuser(), and publishedarticlecountsbyuser() resolve author strings to active users before showing public author data.

477-494 | revisionmatchessearch() applies the same normal text and #tag search behavior to pending revisions.

498-511 | findpublishedarticlebytag() finds the newest public approved article with a reserved page tag. This powers /about and /wiki/<tag>.

515-531 | normalizearticletags() lowercases comma-separated tags, removes duplicates, and blocks normal users from creating reserved page:* tags.

534-543 | caneditarticle() allows edits only for authenticated users, blocks archived articles, and permits admins, writers, or the article owner.

8. Upload helpers, username sync, revisions, and rendering

Lines | What those lines do / why they matter

548-560 | parseinfoboxdata() turns simple Label: Value lines into rows used by the article infobox. Lines without a colon become labels with empty values.

563-611 | savearticleimage(), saveprofileimage(), and savesiteimage() share the safe upload pattern: reject empty filenames, sanitize the original name, keep the extension, generate a UUID filename, save to the correct folder, and return a static/img-relative path.

614-621 | syncusernamereferences() updates article author, approval, and revision editor fields after a username change.

624-644 | saveorreplacependingrevision() keeps one pending revision per article, creating it when needed and copying the submitted form data into it.

647-661 | apply_revision() copies reviewed revision fields onto the live article, marks it approved, records reviewer metadata, and deletes the revision.

664-687 | renderarticlepage() renders markdown, parses infobox rows, finds pending revision status, builds default infobox rows when needed, and renders article.html.

9. Authentication and profile routes

Lines | What those lines do / why they matter

692-716 | register() validates registration, checks duplicate email and username, hashes the password through User.set_password(), commits the new user, and redirects to login.

719-742 | login() validates email/password, blocks archived accounts, calls loginuser(), and redirects through safenext_url().

745-751 | logout() requires login, clears the Flask-Login session, flashes success, and redirects home.

754-804 | profile() hosts two forms on one page: profile edits and password changes. It uses form_type to decide which branch to validate, handles username uniqueness, optional profile image upload, username reference sync, and current-password checks.

10. User pages and public browsing routes

Lines | What those lines do / why they matter

809-845 | my_pages() shows the current user’s articles with search, status, archive visibility, and sort filters. It also prepares article links for the template.

848-869 | authors() lists public accounts. By default it shows users with published articles; all=1 includes all active users.

872-884 | public_profile() shows one active user’s profile and approved public articles. Archived users return 404.

888-890 | index() renders the home page.

894-901 | site_search() routes navbar searches: @name goes to author search, everything else goes to article search.

904-911 | about() renders the published page:about-us article, or redirects home with an info message if none exists.

914-921 | tagged_article() renders a reserved tag-backed wiki page such as /wiki/levels or /wiki/lore.

924-944 | articles() lists approved public articles, filters out articles with inactive authors, applies optional search, and sends links/author data to articles.html.

11. Article creation and editing

Lines | What those lines do / why they matter

949-995 | create_article() validates ArticleForm, checks title uniqueness, normalizes tags, saves an uploaded image or uses default.png, creates a pending Article, commits it, and

redirects to the user’s pages unless it is immediately public.

998-1067 | edit_article() loads an article, blocks archived or unauthorized edits, pre-fills the form from an existing pending revision when present, checks duplicate titles, handles uploaded/removed images, and either updates the live row directly for admins/writers or stores a pending revision for normal users.

1033-1040 | Image selection rules: a new upload wins; otherwise a remove request resets to default.png; otherwise the old image is kept.

1042-1058 | Trusted editors update the live article directly. Normal users submit a pending ArticleRevision for review.

1060-1065 | After commit, image cleanup runs when the image changed, then the user is redirected to the public article if visible or back to my_pages.

12. Dashboard routes

Lines | What those lines do / why they matter

1072-1076 | /dashboard is admin-only and redirects to the user dashboard.

1079-1119 | dashboard_users() is admin-only. It applies search, role, archive, and sort filters; computes published article counts; creates RoleForm; and renders the user dashboard.

1122-1161 | dashboard_articles() is admin/writer-only. It filters articles by archive state, status, search text, and sort order, then prepares public links and author data.

1164-1196 | dashboard_updates() lists pending revisions with search and sorting, then prepares article links for the updates dashboard.

1199-1216 | dashboard_settings() lets admins upload the site/community banner image.

1219-1227 | removeherobanner() clears the banner image from site settings.

13. Admin and reviewer actions

Lines | What those lines do / why they matter

1232-1259 | set_role() changes a user’s role while protecting MainAdmin, archived users, the current admin’s own account, and admin-role boundaries.

1262-1281 | togglearchiveuser() deactivates/reactivates users while blocking MainAdmin, self-archive, and non-MainAdmin archive of other admins.

1284-1306 | approve() approves a new article or applies its latest pending revision, then records approval metadata.

1309-1323 | decline() marks an article declined and records reviewer metadata.

1326-1342 | approve_revision() checks for duplicate titles, applies the revision, commits, and returns to the update dashboard.

1345-1354 | decline_revision() deletes a pending revision without changing the live article.

1357-1366 | togglearchivearticle() lets admins/writers archive or restore articles from the reviewer dashboard.

1369-1384 | archiveownarticle() lets owners archive/restore their own articles; archiving discards pending revisions.

14. Final public article route

Lines | What those lines do / why they matter

1389-1398 | article() loads a non-archived article by exact title, rejects non-public articles, redirects the reserved about page to /about, and renders normal articles through renderarticlepage().

15. Complete function and route index

Name | Lines | Route/decorators | Purpose

cleanupunusedimages | 46-81 | - | Remove uploaded image files no longer referenced by the database.

ensureuserprofile_columns | 84-95 | - | Add profile columns to older SQLite databases.

getsitesettings | 98-105 | - | Return/create singleton site settings row.

handleuploadtoo_large | 127-130 | @app.errorhandler(RequestEntityTooLarge) | Show friendly upload-size error.

injectsitesettings | 134-146 | @app.context_processor | Expose site settings and dashboard counters to templates.

load_user | 150-155 | @loginmanager.userloader | Load active users for Flask-Login sessions.

role_required | 158-171 | - | Decorator factory for role-protected routes.

issaferedirect_url | 174-180 | - | Prevent open redirects.

safenexturl | 183-188 | - | Resolve safe next URL or default endpoint.

dashboardreturnurl | 191-196 | - | Return to safe dashboard filter URL after POST.

inlinemarkdown | 201-222 | - | Render safe inline markdown subset.

rendersimplemarkdown | 225-329 | - | Render safe block markdown subset.

articlepublicurl | 334-338 | - | Resolve normal or reserved article URL.

normalize_identity | 341-344 | - | Normalize author/user names for matching.

finduserforarticleauthor | 347-371 | - | Resolve article author text to active user.

articleispublic | 374-378 | - | Apply public article visibility rules.

canopenarticle | 381-383 | - | Public article-open guard.

staticpagetag | 386-388 | - | Create reserved page tag.

articlehastag | 391-394 | - | Case-insensitive tag lookup.

tagsearchneedle | 397-402 | - | Parse #tag search syntax.

articlematchessearch | 405-422 | - | Match articles by text or tags.

usermatchespublic_search | 425-437 | - | Match users by public fields.

activeusersby_username | 440-447 | - | Resolve active users from author names.

publicarticlesfor_user | 450-460 | - | Return public articles for one user.

publishedarticlecountsbyuser | 463-474 | - | Count public articles by active resolved user.

revisionmatchessearch | 477-494 | - | Match pending revisions by text or tags.

findpublishedarticlebytag | 498-511 | - | Find public article for reserved page tag.

normalizearticletags | 515-531 | - | Normalize tags and protect page:*.

caneditarticle | 534-543 | - | Enforce edit permissions.

parseinfoboxdata | 548-560 | - | Parse infobox rows.

savearticleimage | 563-577 | - | Store article image upload.

saveprofileimage | 580-594 | - | Store profile image upload.

savesiteimage | 597-611 | - | Store site/banner image upload.

syncusernamereferences | 614-621 | - | Update username references after rename.

saveorreplacependingrevision | 624-644 | - | Keep one pending revision per article.

apply_revision | 647-661 | - | Promote revision into live article.

renderarticlepage | 664-687 | - | Build data for article template.

register | 692-716 | /register GET/POST | Create account.

login | 719-742 | /login GET/POST | Authenticate account.

logout | 745-751 | /logout, login required | End session.

profile | 754-804 | /profile GET/POST, login required | Edit profile or password.

my_pages | 809-845 | /my-pages, login required | Show user’s own articles.

authors | 848-869 | /authors | Browse/search public users.

public_profile | 872-884 | /authors/<username> | Show public profile and articles.

index | 888-890 | / | Home page.

site_search | 894-901 | /search | Route search to authors or articles.

about | 904-911 | /about | Render reserved about article.

tagged_article | 914-921 | /wiki/<tag> | Render reserved tag-backed article.

articles | 924-944 | /articles | Browse public articles.

create_article | 949-995 | /articles/new GET/POST, login required | Create pending article.

edit_article | 998-1067 | /articles/<id>/edit GET/POST, login required | Edit live article or submit revision.

dashboard | 1072-1076 | /dashboard, admin | Redirect to dashboard users.

dashboard_users | 1079-1119 | /dashboard/users, admin | Manage users and roles.

dashboard_articles | 1122-1161 | /dashboard/articles, admin/writer | Review and manage articles.

dashboard_updates | 1164-1196 | /dashboard/updates, admin/writer | Review pending updates.

dashboard_settings | 1199-1216 | /dashboard/settings GET/POST, admin | Manage site banner.

removeherobanner | 1219-1227 | /dashboard/settings/banner/remove, admin | Remove site banner.

setrole | 1232-1259 | /set-role/<userid> POST, admin | Change user role.

togglearchiveuser | 1262-1281 | /users/<user_id>/toggle-archive POST, admin | Deactivate/reactivate user.

approve | 1284-1306 | /approve/<article_id> POST, admin/writer | Approve article or latest revision.

decline | 1309-1323 | /decline/<article_id> POST, admin/writer | Decline article.

approverevision | 1326-1342 | /revisions/<revisionid>/approve POST, admin/writer | Approve pending revision.

declinerevision | 1345-1354 | /revisions/<revisionid>/decline POST, admin/writer | Discard pending revision.

togglearchivearticle | 1357-1366 | /articles/<article_id>/toggle-archive POST, admin/writer | Archive/restore article.

archiveownarticle | 1369-1384 | /articles/<article_id>/archive-own POST, login required | Owner archive/restore action.

article | 1389-1398 | /article/<article_title> | Render one public article.

16. Important workflows in plain language

A. Creating a new article

- User opens /articles/new.
- ArticleForm validates title, content, summary, infobox, image, and tags.
- Title uniqueness is checked before insert.
- Tags are normalized; reserved page:* tags are blocked for normal users.
- Image is saved or default.png is used.
- Article is saved with status='pending'.
- A reviewer later approves it before it becomes public.

B. Editing an article

- Archived articles cannot be edited.
- The current user must be admin, writer, or article owner.
- Existing pending revision data is used to prefill the form when present.

- Admins and writers update the live article directly.
- Normal users create or replace one pending ArticleRevision.
- Approved public articles redirect back to their public page after edit.

C. Showing a public article

- Article must exist and not be archived.
- Article status must be approved.
- Author must resolve to an active, non-archived user.
- Content is converted through the safe markdown renderer.
- Template receives rendered content, infobox rows, author data, edit permission, public link, and pending revision information.

D. Admin safety rules

- MainAdmin cannot be archived or have role changed.
- Admins cannot archive themselves or change their own role.
- Only MainAdmin can add or remove admin privileges.
- Archived users cannot log in, and their articles are not public.

17. Security and robustness notes

```
Area | Good practice in this code | Potential improvement
Markdown rendering | Escapes raw HTML and supports only a limited tag set. | Use a tested markdown library plus sanitizer if markdown features grow.
Redirects | Checks host before redirecting to next or referrer. | Continue routing all user-controlled redirects through the safe helpers.
Uploads | Uses secure_filename, UUID names, WTForms extension checks, and file-size checks. | Add MIME/content validation for stronger image-only enforcement.
Secret key | Configured in Flask app config. | Move SECRET_KEY to an environment variable before deployment.
Database migrations | Small SQLite compatibility migration exists. | Use Flask-Migrate/Alembic if schema changes keep growing.
Authorization | Route decorators and ownership checks are explicit. | Review whether writers should approve all content or only scoped content, depending on moderation policy.
```

18. Short glossary

```
Term | Meaning in this app
Article | Main wiki page record. Can be pending, approved, declined, or archived.
ArticleRevision | Pending edit proposal for an existing article.
Archived | Hidden/deactivated but not deleted.
Reserved page tag | A special tag like page:about-us that connects an article to a static route.
Flask-Login | Library that manages logged-in sessions and current_user.
Decorator | Function wrapper placed above a route to add behavior such as login or role checks.
Flash message | Temporary message shown to the user after an action.
```